U24AI047 AI LAB PRACTICAL-11

1. Consider the following districts in the map of Gujarat.

Color this map with the lowest number of colors posing it as a constraint satisfaction problem so that no two adjacent districts are colored the same. [Diu is not a district of Gujarat and Rann of Kuchchh and Little Rann of Kuchchh are also not districts]

```
dist = ["Kuchchh", "Banaskantha", "Patan", "Mehsana",
"Sabarkantha",
        "Gandhinagar", "Ahmedabad", "Kheda", "Panchmahal",
"Dahod",
        "Surendranagar", "Rajkot", "Jamnagar", "Porbandar",
"Junagadh",
```

```
        "Amreli", "Bhavnagar", "Anand", "Vadodara", "Narmada",
        "Bharuch", "Surat", "Navsari", "Valsad", "Dangs"]

# Map city string to its number (index 0 to 24)
city_map = {}
for i in range(len(dist)):
    city_map[dist[i]] = i

# Create empty adjacency matrix (25x25)
n = len(dist)
adj_matrix = []
for i in range(n):
    row = []
    for j in range(n):
        row.append(0)
    adj_matrix.append(row)

# The original adjacency list
adj_list = {
"Kuchchh": [],
 "Banaskantha": ["Patan", "Mehsana", "Sabarkantha"],
 "Patan": ["Banaskantha", "Surendranagar", "Mehsana"],
 "Mehsana": ["Patan", "Banaskantha", "Sabarkantha",
"Gandhinagar",
             "Ahmedabad", "Surendranagar"],
 "Sabarkantha": ["Banaskantha", "Mehsana", "Gandhinagar",
"Kheda",
                 "Panchmahal"],
 "Surendranagar": ["Patan", "Mehsana", "Ahmedabad",
"Bhavnagar",
                   "Rajkot"],
 "Rajkot": ["Surendranagar", "Bhavnagar", "Amreli",
"Junagadh",
                "Porbandar", "Jamnagar"],
 "Jamnagar": ["Rajkot", "Porbandar"],
 "Porbandar": ["Jamnagar", "Rajkot", "Junagadh"],
 "Junagadh": ["Porbandar", "Rajkot", "Amreli"],
 "Amreli": ["Junagadh", "Rajkot", "Bhavnagar"],
 "Bhavnagar": ["Amreli", "Rajkot", "Surendranagar",
"Ahmedabad"],
 "Ahmedabad": ["Surendranagar", "Mehsana", "Gandhinagar",
"Kheda", "Anand",
```

```
"Bhavnagar"],
 "Gandhinagar": ["Mehsana", "Sabarkantha", "Kheda",
"Ahmedabad"],
 "Kheda": ["Gandhinagar", "Sabarkantha", "Panchmahal",
"Vadodara", "Anand",
           "Ahmedabad"],
 "Anand": ["Ahmedabad", "Kheda", "Vadodara"],
 "Panchmahal": ["Sabarkantha", "Kheda", "Vadodara", "Dahod"],
 "Dahod": ["Panchmahal", "Vadodara"],
 "Vadodara": ["Anand", "Kheda", "Panchmahal", "Dahod",
"Narmada", "Bharuch"],
 "Bharuch": ["Vadodara", "Narmada", "Surat", "Anand"],
 "Narmada": ["Vadodara", "Bharuch", "Surat"],
 "Surat": ["Bharuch", "Narmada", "Navsari", "Dangs"],
 "Navsari": ["Surat", "Dangs", "Valsad"],
 "Dangs": ["Navsari", "Surat"],
 "Valsad": ["Navsari"]
}

# Fill the matrix
for city in adj_list:
    u = city_map[city]
    for neighbor in adj_list[city]:
        v = city_map[neighbor]
        adj_matrix[u][v] = 1
        adj_matrix[v][u] = 1 # undirected graph

# Array to store assigned color for each city index
res = [] #colors

# check if assigning color 'c' to city 'node' (index) is safe
def check(node, c):
    # Loop through all 25 cities
    for neighbor in range(n):
        # If they are connected (value is 1 in matrix)
        if adj_matrix[node][neighbor] == 1:
            # If that neighbor already has the color we want
to use, fail
            if res[neighbor] == c:
                return False
    return True
```

```python
# Standard Backtracking (NO Forward Checking)
def run(idx, colors):
    if idx == n:
        return True

    for c in colors:
        if check(idx, c):
            res[idx] = c
            if run(idx + 1, colors):
                return True
            # Backtrack
            res[idx] = -1
    return False

# main loop to find min colors
for i in [1, 2, 3, 4]:
    cols = []
    if i == 1: cols = ["Red"]
    elif i == 2: cols = ["Red", "Blue"]
    elif i == 3: cols = ["Red", "Blue", "Green"]
    else: cols = ["Red", "Blue", "Green", "Yellow"]

    # Initialize array with -1 meaning "uncolored"
    res = []
    for _ in range(n):
        res.append(-1)

    if run(0, cols):
        print("Minimum colors used:", i)
        print("No. District \t Color")
        for k in range(n):
            print(str(k) + "   " + dist[k], "\t", res[k])
        break
```

```
param@GREYRAT MINGW64 /c/SVNIT PRACTICALS/AI-lab
$ python -u "c:\SVNIT PRACTICALS\AI-lab\LAB-11\Q1.py"
Minimum colors used: 3
No. District      Color
0   Kuchchh       Red
1   Banaskantha           Red
2   Patan         Blue
3   Mehsana       Green
4   Sabarkantha           Blue
5   Gandhinagar           Red
6   Ahmedabad     Blue
7   Kheda         Green
8   Panchmahal    Red
9   Dahod         Green
10   Surendranagar        Red
11   Rajkot       Blue
12   Jamnagar     Green
13   Porbandar    Red
14   Junagadh     Green
15   Amreli       Red
16   Bhavnagar    Green
17   Anand        Red
18   Vadodara     Blue
19   Narmada      Red
20   Bharuch      Green
21   Surat        Blue
22   Navsari      Red
23   Valsad       Blue
```

2. Solve the following cryptarithmetic problem posing it as a constraint satisfaction problem.

Each letter represents a unique digit from 0 to 9. The leading letters (S and M) cannot be

Solve:

```
   SEND
+  MORE
---------------
   MONEY
```

```
def solve_send_more_money():
    variables = ['D', 'E', 'Y', 'N', 'R', 'O', 'S', 'M']
    all_solutions = []
```

```
    def is_consistent(var, value, assignment):
        if value in assignment.values():
            return False
        if var in ['S', 'M'] and value == 0:
            return False
        return True

    def backtrack(assignment):
        c1 = c2 = c3 = c4 = 0

        # Early pruning via column-by-column carry checks
        if all(k in assignment for k in ('D', 'E', 'Y')):
            if (assignment['D'] + assignment['E']) % 10 != assignment['Y']: return
            c1 = (assignment['D'] + assignment['E']) // 10

            if all(k in assignment for k in ('N', 'R')):
                if (assignment['N'] + assignment['R'] + c1) % 10 !=
assignment['E']: return
                c2 = (assignment['N'] + assignment['R'] + c1) // 10

                if 'O' in assignment:
                    if (assignment['E'] + assignment['O'] + c2) % 10 !=
assignment['N']: return
                    c3 = (assignment['E'] + assignment['O'] + c2) // 10

                    if all(k in assignment for k in ('S', 'M')):
                        if (assignment['S'] + assignment['M'] + c3) % 10 !=
assignment['O']: return
                        c4 = (assignment['S'] + assignment['M'] + c3) // 10
                        if assignment['M'] != c4: return

        # Base case
        if len(assignment) == len(variables):
            all_solutions.append(assignment.copy())
            return

        current_var = variables[len(assignment)]

        for value in range(10):
            if is_consistent(current_var, value, assignment):
                assignment[current_var] = value
```

```python
                backtrack(assignment)
                del assignment[current_var]

    backtrack({})
    return all_solutions

if __name__ == '__main__':
    solutions = solve_send_more_money()

    print(f"Total solutions found: {len(solutions)}\n")

    for idx, solution in enumerate(solutions, 1):
        print(f"Solution {idx}:")
        print(f"  {solution['S']} {solution['E']} {solution['N']} {solution['D']}")
        print(f"+ {solution['M']} {solution['O']} {solution['R']} {solution['E']}")
        print("-" * 9)
        print(f"{solution['M']} {solution['O']} {solution['N']} {solution['E']}
{solution['Y']}\n")
```

```
param@GREYRAT MINGW64 /c/SVNIT PRACTICALS/AI-lab
$ python -u "c:\SVNIT PRACTICALS\AI-lab\LAB-11\Q2_best.py"
Total solutions found: 1

Solution 1:
  9 5 6 7
+ 1 0 8 5
---------
1 0 6 5 2
```